

SECURITY POLICY EXCEPTION REQUEST ANALYZER

Documentation & Workflow Guide

AI-Driven Exception Severity Classification System Using Agentic AI
for Automated Risk Evaluation

Version	1.0
Date	December 8, 2025
Authors	Connor Griffith
Department	UD Information Security
Contact Email	udinfosec@udel.edu

The Policy Exception Request Analyzer automates the review of policy exception requests by using agentic AI to assess severity risk, and compliance impact. This documentation outlines the system design, AI reasoning workflow, governance model, and integration with human review processes

Table Of Contents

1. Overview	3
1.1 Purpose	3
1.2 Goals	3
1.3 Key Benefits	3
1.4 System Context	4
2. Objectives & Scope	4
2.1 Objectives	4
2.2 Scope	5
2.3 Future Scope Considerations	5
3. System Architecture	5
3.1 Architecture Overview	5
3.1.1 TDX Intake and Processing Pipeline	6
3.2 Core Components	6
3.3 Storage and Retrieval Layer	7
3.4 Data Flow	8
4. Workflow Diagram	12
4.1 Overview of Process	12
4.2 Key Workflow Roles	13
5. Input and Output Specifications	14
5.1 Input Fields	14
5.2 Output Fields	14
5.3 Data Flow Between Modules	14
5.4 Data Security Considerations	14
6. AI Decision Logic	14
6.1 Reasoning Process Overview	15
6.2 Severity Scoring Framework	15
6.3 Reasoning Transparency and Explainability	15
7. Human-in-the-Loop Governance	15
7.1 Governance Philosophy	16
7.2 Roles and Responsibilities	16
7.3 Review and Override Process	16
7.4 Ethical and Legal Compliance	17
8. Compliance & Audit Controls	17
9. Example Use Case	17
10. Future Enhancements	17
11. References, Libraries, and Licenses	17

1. Overview

Our project, **The Security Policy Exception Request Analyzer** is an internal AI-assisted ticketing system that streamlines how policy exception requests are reviewed, classified, and escalated. The system uses agentic AI reasoning to automatically assess the **severity** and **risk impact** of requests within UD's system, generating transparent, auditable recommendations for compliance teams.

1.1 Purpose

This document describes how our system functions, the reasoning workflow behind the severity assessments, and how it integrates with human review and governance processes. It is intended for use by the University of Delaware, as well as system owners, compliance officers, security analysts, and developers responsible for maintaining the analyzer.

1.2 Goals

- Automate severity scoring for incoming policy exception requests
- Reduce manual workload for reviewers while improving consistency
- Provide a chatbot for smooth user engagement and experience
- Ensure explainable AI-based reasoning for every decision
- Provide clear escalation paths for high-risk or critical cases
- Maintain full traceability and audit readiness across all actions

1.3 Key Benefits

Benefit	Description
Speed	Reduces the exception processing time by automatically identifying low-risk requests
Consistency	Applies uniform evaluation logic across all policy categories
Transparency	Generates an AI rationale with every recommendation
Governance Alignment	Ensures compliance with internal review policies and risk management standards

1.4 System Context

When a new request is submitted, the analyzer:

1. Retrieves context about the policy, requester, and historical precedents.
2. Applies its reasoning engine to determine a severity score
3. Routes the output and rationale to the appropriate human reviewer

2. Objectives & Scope

The **Policy Exception Request Analyzer** introduces a structured, AI-assisted workflow for evaluating and approving policy exceptions across all departments throughout University of Delaware's infrastructure.

This section defines what the system is designed to achieve and the limits of its current implementation

2.1 Objectives

The primary objectives of the analyzer are to:

- **Automate Severity Assessment:** Use agentic AI to determine the potential risk and impact level of each policy-exception request.
- **Enhance Decision Consistency:** We apply uniform logic regardless of department based off of numerous previous cases submitted
- **Improve Operational Efficiency:** Reduce manual workload and accelerate approval timelines
- **Increase Transparency:** Provide auditable AI-generated rationales for each classification
- **Provide a Seamless User Experience:** Provide users an in-site assistive chatbot to speed up form completion time for confused users
- **Support Governance Oversight:** Enable human reviewers to focus on high-impact exceptions while maintaining traceability and control

2.2 Scope

In Scope:

The analyzer currently supports requests and policy categories related to:

- Security policy exception requests submitted through TDX
- Device, system, and infrastructure exceptions
- Exceptions involving hostnames, public IP exposure, management network access, patching practices, and firewall rules
- Requests involving data classification, data access level, and business justification
- Exceptions requiring review of compensating controls and mitigation statements
- Attachments submitted with tickets, including CSV, TXT, DOCX, XLSX, PDF, and image files for supplemental context

Out of Scope:

The analyzer does **not** currently evaluate:

- Real-time threat detection or incident response activity
- Automated remediation or security control enforcement
- General IT service requests unrelated to security policy exceptions
- Full malware analysis or deep forensic analysis of uploaded attachments
- Final approval authority without human review for high-risk requests

2.3 Future Scope Considerations

Planned Expansion areas include:

- Integration with third-party risk management tools
- Cross-policy correlation for compound risk scenarios
- Multilingual support for international universities or organizations
- Adaptive learning based on reviewer feedback loops

3. System Architecture

The **Policy Exception Request Analyzer** consists of a pipeline combining data ingestion, agentic AI reasoning, and workflow orchestration. Each component plays a specific role in evaluating requests, assessing severity levels, and producing audit-ready recommendations

3.1 Architecture Overview

At a high level, the analyzer operates across five stages:

1. **Request Intake:** Collects exception requests via a UD portal and exception intake form.
2. **Context Retrieval:** Gathers relevant policy text, requester profile, and historical precedents previously fed to the RAG system.
3. **AI Reasoning Engine:** Applies contextual reasoning using Google Gemini's-based logic to assess severity.
4. **Recommendation Engine:** Generates recommended actions and routing paths
5. **Audit and Storage Layer:** Records all actions, rationales and outcomes for human compliance review

3.1.1 TDX Intake and Processing Pipeline

The Policy Exception Request Analyzer uses TeamDynamix (TDX) as the entry point for exception requests. The backend continuously polls the TDX workflow API for open tickets and processes only newly discovered ticket IDs. Each ticket is normalized into a structured JSON object that can be passed into downstream analysis and storage components.

The TDX processing flow works as follows:

1. A requester submits a policy exception request through the TDX form.
2. The backend calls the TDX workflow API using the Get_Tickets method with the Only_Open=true parameter to retrieve open requests.
3. Newly detected ticket IDs are compared against a local cache to avoid duplicate processing.
4. For each new ticket, the system calls Get_Ticket to retrieve the full ticket payload.
5. Relevant ticket metadata is extracted, including requester name, department, description, and mapped form attributes.
6. If attachments are present, the system calls Get_Attachment, decodes the base64 file content, saves the file locally, and attempts lightweight interpretation based on file type.
7. The resulting normalized request object is cached and made available to downstream AI reasoning, policy retrieval, and decision-support modules.

3.2 Core Components

Component	Function	Key Technologies/Sources
Request Intake Module	Collects exception requests from TDX, normalizes ticket data, extracts key fields, and prepares requests for downstream analysis.	TeamDynamix (TDX) API, Python Requests Library
Context Retriever Agent	Searches indexed policy content and historical exception context to retrieve the most relevant governance material for the submitted request	.RAG Pipeline, Firestore Vector Search
Vector Database	Stores policy embeddings and metadata for semantic retrieval and similarity matching.	Google Firestore
Embedding Model	Converts policy text and requests into vector embeddings for semantic search.	Google Gemini embedding API (768-dimensional vectors)
Language Model	Performs contextual reasoning, compliance analysis, and recommendation generation.	Google Gemini 2.0-flash (non-reasoning model)
Policy Corpus	Repository of university security policies and mapped governance references.	79 university security policies with NIST SP 800-53 references
Risk Scoring Module	Applies weighted scoring logic to request characteristics, security controls, data classification, and policy conflicts to generate a severity score.	Rule-Based Scoring Engine, Python Logic
Recommendation Engine	Combines retrieved policy context, risk score, and model reasoning to produce a recommended action	Gemini Prompting, Decision Logic

	such as approve, escalate, or reject	
Human Review Interface	Presents AI outputs, supporting rationale, and requests details to reviewers so they can validate, override, or finalize the decision.	reports/ directory (plain-text .txt report per request)

3.3 Storage and Retrieval Layer

Google Firestore serves as the primary document-oriented storage layer for the analyzer's Retrieval-Augmented Generation workflow. In the broader system design, Firestore stores policy chunks, embeddings, and associated metadata used during semantic retrieval. The documentation already notes that vector storage occurs in the **policies** namespace, where policy chunks are stored with metadata such as policy category, identifiers, and NIST mappings.

Firestore supports the analyzer in three main ways:

- **Policy chunk storage:** security policies are broken into searchable chunks and stored with metadata.
- **Vector retrieval support:** embeddings generated from policy text are associated with each chunk to enable semantic similarity search.
- **Audit and traceability support:** processed requests, AI outputs, and reviewer actions can be recorded in a structured format for later review.

Firestore was selected because it supports flexible schemas, integrates well with Google Cloud services, and can support both metadata-rich document storage and retrieval workflows needed for RAG-based analysis.

3.4 Data Flow

Complete RAG Pipeline

Phase 1: Data Ingestion & Indexing (Pre-Processing)

- University Policy Documents → Text Extraction → Chunking → Embedding Generation → Vector Storage

Inputs:

- University security policy documents (PDF, Word, text)
- Policy metadata (categories, NIST references, effective dates)

Processing Steps:

1. **Document Parsing:** Extract text content from policy documents
2. **Text Chunking:** Break documents into semantic chunks
3. **Metadata Enrichment:** Add policy categories, NIST mappings, and identifiers
4. **Embedding Generation:** Create 768-dimensional vectors using Google Gemini embedding API
5. **Vector Storage:** Store embeddings in Firestore with metadata in '**policies**' namespace

Outputs:

- Searchable vector database with 79 indexed policies
- Metadata-enriched policy chunks with semantic relationships

Phase 2: Exception Request Processing (Runtime)**Step 2.1: Policy Search & Retrieval**

- Exception Request → Query Formation → Hybrid Search → Relevant Policies
- Function: ``hybrid_search(query: str, top_k: int = 5)``

Inputs:

- Natural language query describing the exception request
- Search parameters (top_k results, filters)

Processing Steps:

1. **Query Processing:** Clean and normalize input text
2. **Embedding Generation:** Convert query to 768-dimensional vector
3. **Hybrid Search Execution:**
 - Vector similarity search in Firestore
 - Keyword matching for specific policy references
 - Relevance score calculation (0.0 - 1.0 scale)
4. **Result Ranking:** Order by relevance score and metadata matching

Outputs:

- List of ``PolicyMatch`` objects containing:
 - Policy ID and text content
 - Relevance score
 - Policy metadata (category, NIST references)

Step 2.2: Compliance Analysis

- Exception Request + Retrieved Policies → LLM Analysis → Compliance Assessment
- Function: ``policy_compliance_checker(exception_request: Dict, top_k: int = 5)``

Inputs:

- Exception request details:

```

json
{
  "id": "EXC-2025-001",
  "exception_type": "cloud database hosting",
  "data_level": "Level III",
  "security_controls": ["encryption at rest", "VPN access"],
  "description": "Migration to AWS RDS",
  "business_justification": "Cost reduction",
  "duration": "permanent",
  "affected_systems": ["payroll-db"]
}

```
- Search parameters for policy retrieval

Processing Steps:

1. **Policy Retrieval:** Use hybrid search to find relevant policies
2. **Context Assembly:** Combine exception details with retrieved policies
3. **LLM Prompt Construction:** Create structured prompt for compliance analysis
4. **LLM Analysis:** Send to Google Gemini for policy interpretation
5. **Response Parsing:** Extract structured compliance assessment

Result Validation: Ensure response completeness and format

Outputs:

- Compliance assessment object:

```

json
{
  "compliance_status": "NON_COMPLIANT | POTENTIAL_ISSUE | COMPLIANT",
  "violations": ["List of specific policy violations"],
  "required_controls": ["List of mandatory security controls"],
  "policy_refs": ["Referenced policy IDs"],
  "analysis_summary": "Executive summary text"
}

```

Step 2.3: Risk Narrative Generation

- Risk Factors + Policy Context → LLM Synthesis → Executive Report
- Function: `generate_risk_narrative(risk_score: float, factors: Dict, policy_refs: List)`

Inputs:

- Quantitative risk score (approximately -20 to 114; higher score = stronger security posture)
- Risk factor details:

```

json
{

```

```

"data_classification": "Level III (PII, Financial)",
"environment": "Public Cloud (AWS)",
"compliance_requirements": ["SOX", "GDPR"],
"existing_controls": ["Encryption", "Access logging"],
"missing_controls": ["DLP", "Real-time monitoring"],
"business_impact": "High - Critical HR operations",
"implementation_timeline": "6 months"
}

```

- Applicable policy references

Processing Steps:

1. **Context Aggregation:** Combine risk data with policy requirements
2. **Narrative Template Selection:** Choose appropriate executive report format
3. **LLM Generation:** Create comprehensive risk narrative
4. **Executive Formatting:** Structure for management consumption
5. **Quality Validation:** Ensure professional tone and completeness

Outputs:

- Executive risk narrative containing:
 - Risk assessment summary
 - Business impact analysis
 - Recommended mitigations
 - Policy compliance guidance
 - Implementation priorities

Phase 3: Decision Support Integration

Step 3.1: Risk Score Enhancement

- Compliance Results → Risk Factor Adjustment → Enhanced Risk Score

Integration Points:

- Compliance results are produced separately from the rule-based approval score and do not adjust it. The two assessments are independent outputs.

Step 3.2: Approval Workflow Routing

- Risk Assessment → Business Rules → Workflow Assignment

Decision Logic:

- High-risk exceptions → Senior management approval
- Policy violations → Security committee review
- Compliant requests → Automated processing

Step 3.3: Executive Reporting

- Risk Narratives → Report Generation → reports/ directory (.txt files)

Report Components:

- Executive summary with risk scoring
- Policy compliance assessment
- Recommended security controls
- Implementation timeline and priorities

In Summary...

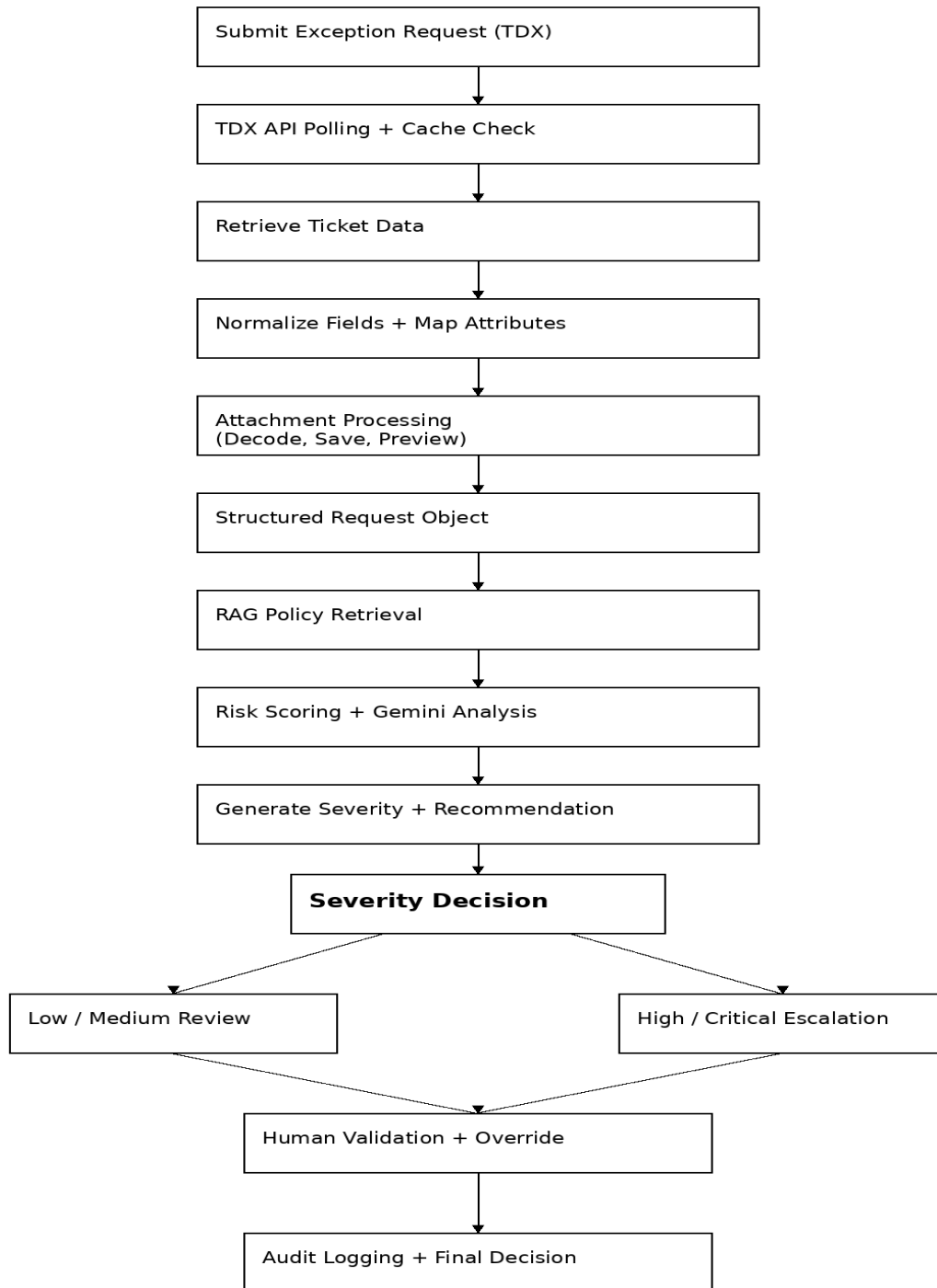
1. **Request Submitted:** User submits exception form
2. **Data Ingested:** request Intake Module normalizes inputs
3. **Context Retrieved:** Retriever Agent fetches metadata and prior decisions
4. **AI Analysis:** Gemini evaluates risk factors and assigns scores, assisting an existing hard coded scoring system
5. **Recommendation Generated:** Engine outputs severity label and rationale
6. **Review and Decision:** High-risk cases routed to a human receiver; low-risk is auto-approved for compliant cases
7. **Logging:** Audit logger captures full decision record for compliance reasons

4. Workflow Diagram

This section illustrates the end-to-end flow of a policy exception request as it moves through the analyzer's components. The workflow highlights how agentic AI collaborates with different software and human reviewers to ensure consistent, explainable, and auditable decisions

4.1 Overview of Process

The diagram below summarizes the major stages:

Policy Exception Analyzer Workflow

This linear-plus-branch workflow ensures

- Speed for low-risk, auto classified exceptions
- Transparency for high-risk cases that require human validation
- Full traceability

4.2 Key Workflow Roles

Role	Responsibility
Requester	Submits the exception request, provides justification, and includes required business/technical details.
Agentic AI Analyzer	Retrieves policy context, evaluates request attributes, assists scoring, and generates recommendations with supporting rationale
Compliance Reviewer	Validates AI-assisted scoring outputs. Can override decisions and review final actions.

5. Input and Output Specifications

This section defines the data elements used by the **Policy Exception Request Analyzer**, including all request inputs, AI-generated outputs, and how information flows between modules. Properly defining these fields ensure traceability, accuracy, and compliance alignment

5.1 Input Fields

Ticket ID: unique identifier retrieved from TDX

Requestor Name: person submitting the request

Department / Account Name: requester's organizational unit

Description / Reason: free-text explanation of the request

Type of Exception: category of requested exception

Exception Start Date: requested effective date

Hostnames: systems covered by the request

Unit Head: organizational approver or responsible owner

Risk Assessment Justification: business and security rationale

Level of Data Stored: classification of data hosted or processed by the asset

Level of Data Accessed: classification of data reachable by the asset

Vulnerability Scanner Allowed: whether scanning agents are permitted
EDR Allowed: whether CrowdStrike or equivalent endpoint tooling is permitted
Local Firewall Rules: local host firewall information
Network Firewall Rules: network-layer restriction information
Management Network Access: whether the system can access management infrastructure
Public IP Address: whether the system is internet-reachable
OS Patch Status: whether the operating system is current
OS Patch Frequency: cadence of operating system updates
Application Patch Frequency: cadence of application updates
Dependency Level: how many assets or services depend on the system
User Impact: how many users are affected by the service
University Impact: overall institutional importance of the asset
Impacted Systems, Services, and Data: business and technical scope of impact
Compensating Controls / Mitigation: summary of safeguards already in place
Attachments: optional uploaded files that provide supporting evidence or documentation

5.2 Output Fields

Normalized Request Object: structured JSON version of the TDX ticket
Extracted Ticket Metadata: requester, department, description, and mapped form fields
Attachment Summaries: file name, file type, storage path, and extracted preview data when available
Retrieved Policy Matches: top relevant policy chunks and metadata from the RAG system
Compliance Assessment: policy alignment result, violations, required controls, and references
Approval Score: numeric value produced by rule-based scoring logic; higher score = stronger security posture (score > 90 auto-approve, < 16 auto-deny)
Severity Category: Low, Medium, High, or Critical
Recommendation: approve, escalate, reject, or require further review
AI Rationale: explanation of why the request received its score and recommendation
Decision Status: pending, under review, approved, rejected, or overridden
Audit Record: timestamped log of AI output and human review actions

5.3 Data Flow Between Modules

1. **The Intake Module** polls TDX for open tickets and identifies new ticket IDs using a local cache.
2. **The Ticket Processing Module** retrieves the full ticket details and maps selected TDX attributes into a normalized request object.
3. **The Attachment Processing Module** retrieves any uploaded files, decodes them from base64, saves them locally, and generates lightweight summaries based on file type.

4. **The Context Retrieval Module** uses the normalized request to query the policy vector store and gather relevant policy and exception context.
5. **The AI Analysis Module** combines the request payload, retrieved context, and rule-based scoring signals to generate severity, rationale, and recommended actions.
6. **The Human Review Layer** receives the output for validation, override, escalation, or final decision.
7. **The Audit and Storage Layer** records the processed request, AI output, and reviewer actions for traceability and compliance.

5.4 Data Security Considerations

- API authentication is enforced through a TDX API key loaded from environment variables rather than hardcoded credentials.
- Ticket data is limited to relevant security exception fields before downstream use, reducing unnecessary data exposure.
- Attachments are stored in ticket-specific directories to preserve separation and traceability.
- File handling includes type-aware parsing, but unsupported file types are saved without advanced analysis to reduce unnecessary processing risk.
- Cached ticket IDs prevent repeated processing and help maintain integrity in the intake workflow.
- Sensitive request data should be protected at rest through secure storage controls and access restrictions.
- All production deployments should restrict filesystem access, protect environment variables, and ensure encrypted transport for all API communications.
- Human reviewers retain oversight of high-risk decisions to reduce the risk of fully automated misclassification.

6. AI Decision Logic

The **Agentic Reasoning Engine** analyzes contextual data and justification text to determine the severity level of each policy-exception request. It uses a hybrid approach combining **semantic understanding**, **rule-based scoring**, and **context-aware reasoning** drawn from past examples and UD's organizational risk policies.

6.1 Reasoning Process Overview

- **Context Retrieval:** Utilizes a Retrieval Augmented Generation (RAG) system to match a user's exemption request to related security policy.
- **Feature Extraction:** Identifies key risk indicators within a user's justification (Ex. mentions of "sensitive data", "Firewall", "urgent access", etc.).
- **Risk Weighting:** Assigns severity levels on a scale of 100 to extracted features based on importance
- **Severity Classification:** Uses prompt-based reasoning and model logic to map numerical risk into categorical severity (Low → Critical).
- **Recommendation Generation:** Produces an action suggestion (Approve, Escalate, Reject) and provides rationale text.

6.2 Severity Scoring Framework

Criterion	Weight	Description/ Examples
Data Sensitivity	High	Requests involving higher-classification institutional, financial, research, or personally identifiable data receive higher risk scores.
Public Exposure	High	Systems with public IP addresses or internet-facing services receive elevated scores due to increased attack surface.
Management Network Access	High	Access to management infrastructure raises severity because compromise could affect administrative or privileged systems.
Patch Posture	High	Systems that are not up to date, or that patch infrequently, increase the likelihood of exploitation.
Endpoint Security Coverage	Medium to high	If vulnerability scanning or EDR tooling is not allowed, the request receives additional risk weight.

Firewall Restrictions	Medium	Weak or missing host/network firewall controls increase exposure and reduce containment.
Dependency Level	Medium	Assets supporting many other systems receive higher impact scores because downstream service disruption is more significant.
User Impact	Medium	Requests affecting larger user populations are weighted more heavily.
University Impact	High	Assets deemed important to the University as a whole receive a stronger weighting due to business criticality.
Compensating Controls	Risk-reducing factor	Strong mitigation measures can lower the final score when well documented and technically credible.
Business Justification Quality	Medium	Well-supported justifications may reduce uncertainty, while vague or incomplete justifications may increase review severity.

6.3 Reasoning Transparency and Explainability

The analyzer provides explainable outputs so reviewers can understand why a request received a specific severity score. Each recommendation includes the main risk factors, relevant policy matches, missing controls, and a short AI-generated rationale. This helps reviewers validate the decision, identify possible errors, and override the result when needed. Transparency is important because the system supports human decision-making but does not replace final human approval.

7. Human-in-the-Loop Governance

Human oversight remains a core element of the Policy Exception Request Analyzer to ensure fairness, accountability, and contextual awareness when Google Gemini makes its decisions. This section describes the roles, responsibilities, and review procedures that maintain compliance and trust within the system.

7.1 Governance Philosophy

- **AI classifies, but humans approve**
- Automated actions mainly occur only for “low-risk” scenarios
- High-impact cases will always require a human decision maker within the exception process

7.2 Roles and Responsibilities

Role	Responsibilities	Access Level
Requester	Submits policy exception request and provides justification	Submit/View own requests
AI Analyzer	Performs contextual reasoning and initial severity scoring	System process
Compliance Reviewer	Validates AI-generated outputs. Has power to override with justification	Read/Write
Manager Approver	Approves/rejects cases that are routed for secondary review	Approve/Comment

7.3 Review and Override Process

- 1. AI Output Received:**
The system classifies each request with severity, rationale, and recommendation.
- 2. Automatic Routing:**
 - a. Low/Medium: Sent to designated reviewer
 - b. High/Critical: Escalated directly to compliance team

3. Human Review:

Reviewers assess the AI rationale, supporting evidence, and contextual details

4. Override Mechanism:

If a reviewer disagrees, they can change the severity level or reject/approve with a justification comment

5. Feedback Loop:

All overrides are logged and periodically analyzed to improve AI accuracy and fairness

7.4 Ethical and Legal Compliance

- **Human Accountability:** Final decision-making authority will always reside within a person
- **Transparency:** Reviewers can view the reasoning factors contributing to severity
- **Privacy:** All reviewer notes and AI outputs follow University of Delaware's data-protection standards

8. Compliance & Audit Controls

To maintain regulatory trust and governance integrity, all outputs of the Policy Exception Request Analyzer are logged, traceable, and subject to formal audit. This section defines how compliance oversight, audit mechanisms, and data protection are built into the system lifecycle

9. Example Use Case

A department submits a policy exception request for a server that needs temporary public IP access while hosting Level III data. The analyzer retrieves the ticket from TDX, normalizes the fields, reviews the business justification, and searches relevant UD security policies through the RAG system. Because the request involves sensitive data and public exposure, the system assigns a High severity rating and recommends escalation to a compliance reviewer. The reviewer then evaluates the AI rationale, checks the compensating controls, and either approves, rejects, or overrides the recommendation with justification.

10. Future Enhancements

As the Policy Exception Request Analyzer continues to evolve, several enhancements are planned to expand its capabilities, improve accuracy, and strengthen governance alignment. This section outlines the upcoming roadmap initiatives.

11. References, Libraries, and Licenses

[Senior Design License & Libraries](#)